

Computergrafik

Vorlesung 1

Hochschule Zittau/Görlitz

Christopher-Manuel Hilgner



Kontakt Daten

- E-Mail: christopher.hilgner@stud.hszg.de
- GitHub: <https://github.com/XPromus>

Prüfung

Schriftliche Klausur

- 120 Minuten
- Themen aus den Vorlesungen und Übungen
- Verständnisfragen
- Grundbegriffe sollten bekannt und erklärbar sein

Inhalte

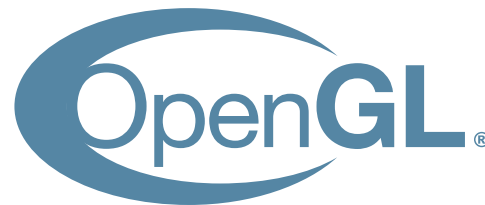
Themen der Vorlesung

- Rendering: Rendering Pipeline, Raytracing, Radiosity, Volume Rendering
- Licht & Schatten
- Partikelsysteme
- Optimierung
- Computeranimationen
- Hardware
- Mathematische Bestandteile der Computergrafik

Inhalte

Themen des Seminars

- Blender
 - 3D Modellierung, Animationen, Rendering
- OpenGL in Java mit [LWJGL](#)
- Ergänzungen zu Unity
- Vielleicht auch etwas [Vulkan](#)



Literatur



Computergrafik

Band I des Standardwerks Computergrafik und
Bildverarbeitung

A. Nischwitz, M. Fischer, P. Haberäcker, G. Socher

[Link zur Springer Website ⇒](#)

Literatur

- A. Nischwitz, M. Fischer, P. Haberäcker & G. Socher (Hrsg.): Computergrafik. eXamen.press, Springer Vieweg, 2019 (4. Auflage).
- Akenine-Möller, Haines, Hoffmann, Pesce, Iwanicki & Hillaire. Real Time Rendering. 5. Auflage, CRC Press. 2018.
- Marschner & Shirley. Fundamentals of Computer Graphics. 4. Auflage, Taylor & Francis Ltd, 2016.

Agenda

Agenda

Vorlesung

- Einführung in die Computergrafik
- Historische Entwicklung
- Rendering Methoden
- Linien Rendering
- Polygone
- Übersicht über die Rendering Pipeline
- Kurze Einführung in Anti-Aliasing
- Anwendungen der Computergrafik

Seminar

- Installation der nötigen Tools
- Erstellung eines einfachen LWJGL Projektes

Einführung

Begriff der *Computergrafik*

Der Begriff *Computergrafik*

- geht zurück auf William Fetter (1928-2002), 1960
- Name für neue Konstruktionsmethoden, die er bei Boeing verfolgte
- Fetter, der das Cockpit-Design erforschte, schuf auf einem Stiftplotter mit Hilfe eines 3D-Modells eines menschlichen Körpers weitgehend reproduzierte Bilder



Figure 1: William A. Fetter, als er bei Boeing arbeitete.

Begriff der *Computergrafik*

Boeing Man

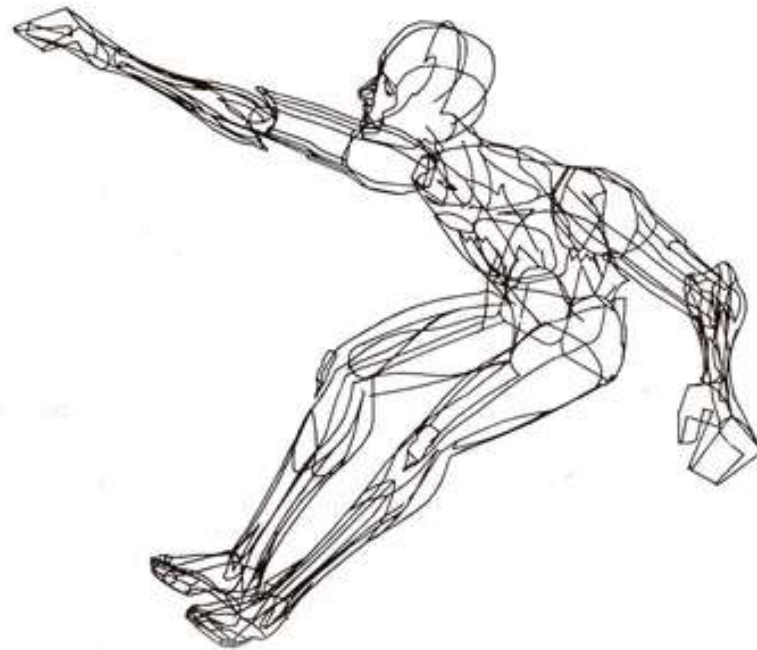


Figure 2: Der “Boeing Man”. Eine Wireframe Darstellung eines Piloten, gedruckt auf einem Gerber Plotter.

Begriff der *Computergrafik*

Definition

Computergrafik

Definition 1

Die Computergrafik ist ein Teilgebiet der Informatik, das sich mit der computergestützten Bilderzeugung, im weiteren Sinne auch mit der Bildbearbeitung befasst. Mit den Mitteln der Computergrafik entstandene Bilder werden Computergrafiken genannt.

Entwicklung der Computergrafik 🚫

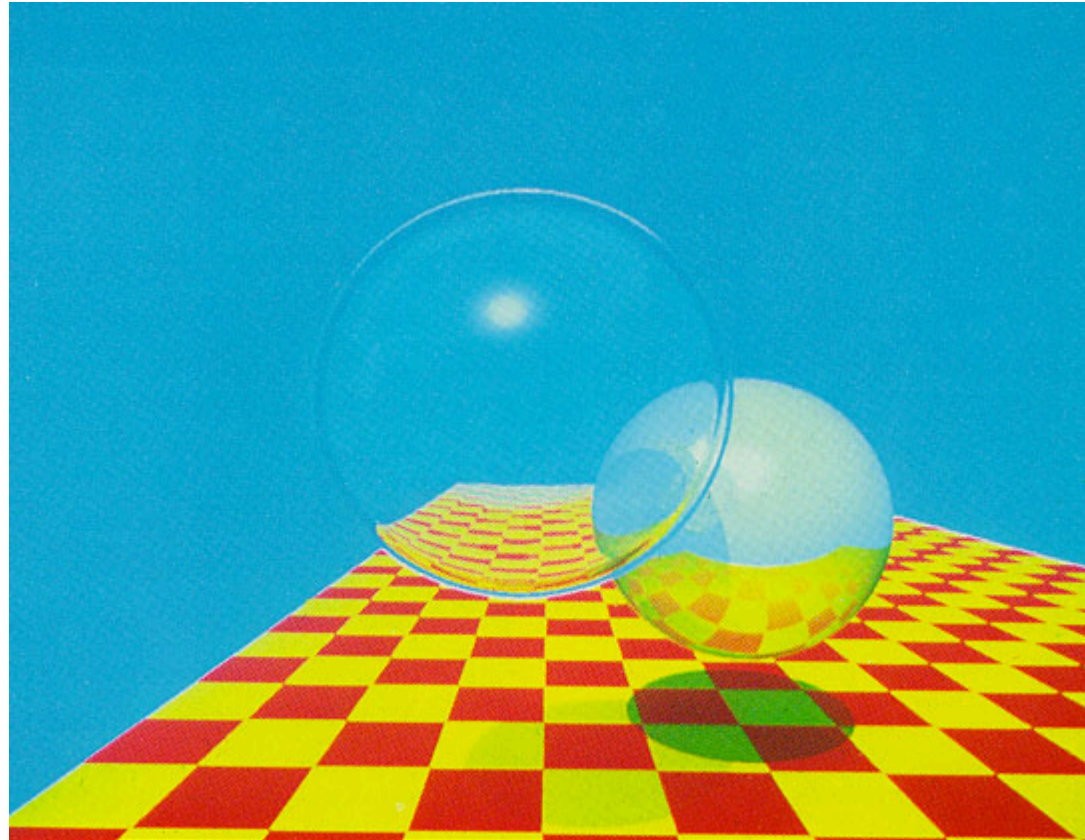


Figure 3: Eine der ersten Computergrafiken mit Lichtspiegelungs- und Brechungseffekten (1980)

Entwicklung der Computergrafik

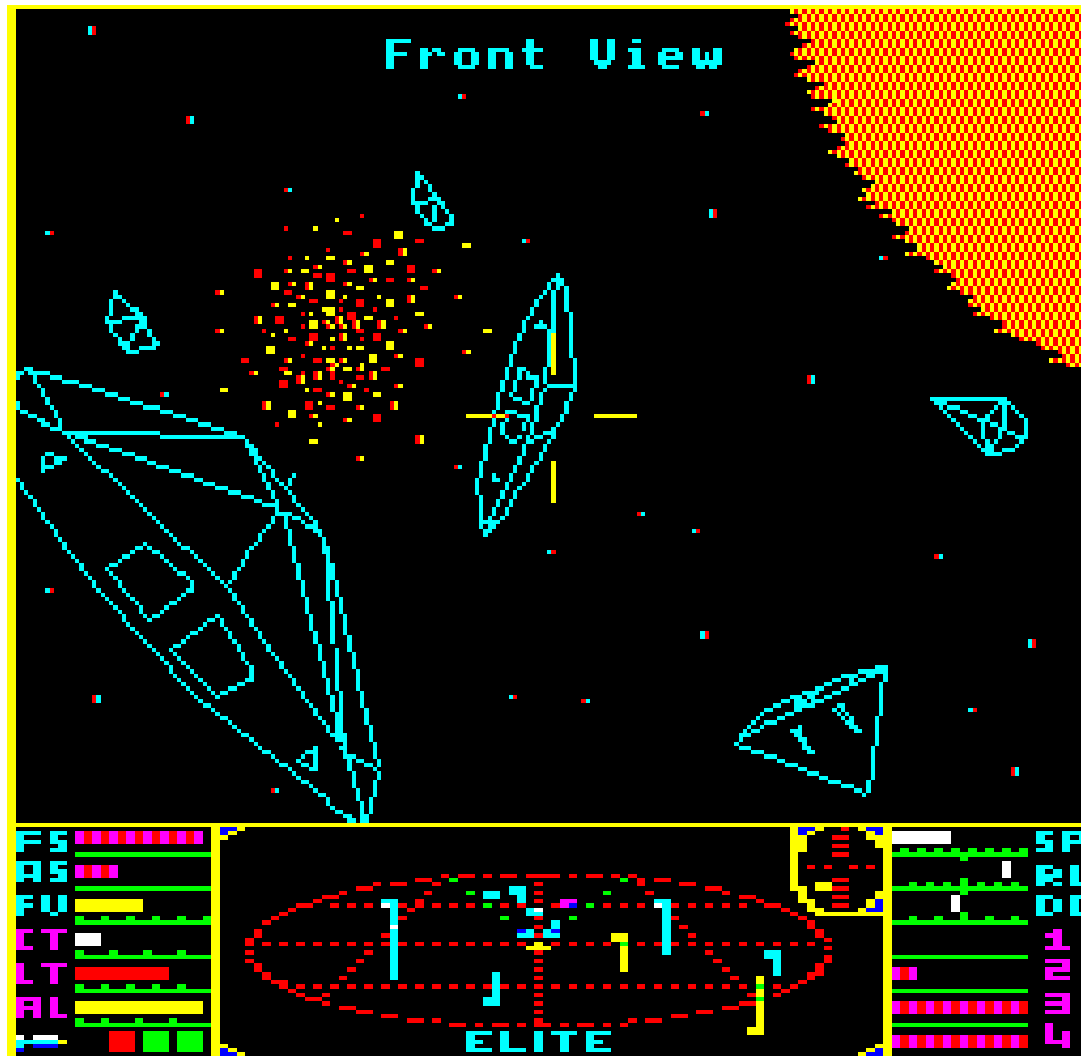


Figure 4: Elite (1984)

Entwicklung der Computergrafik

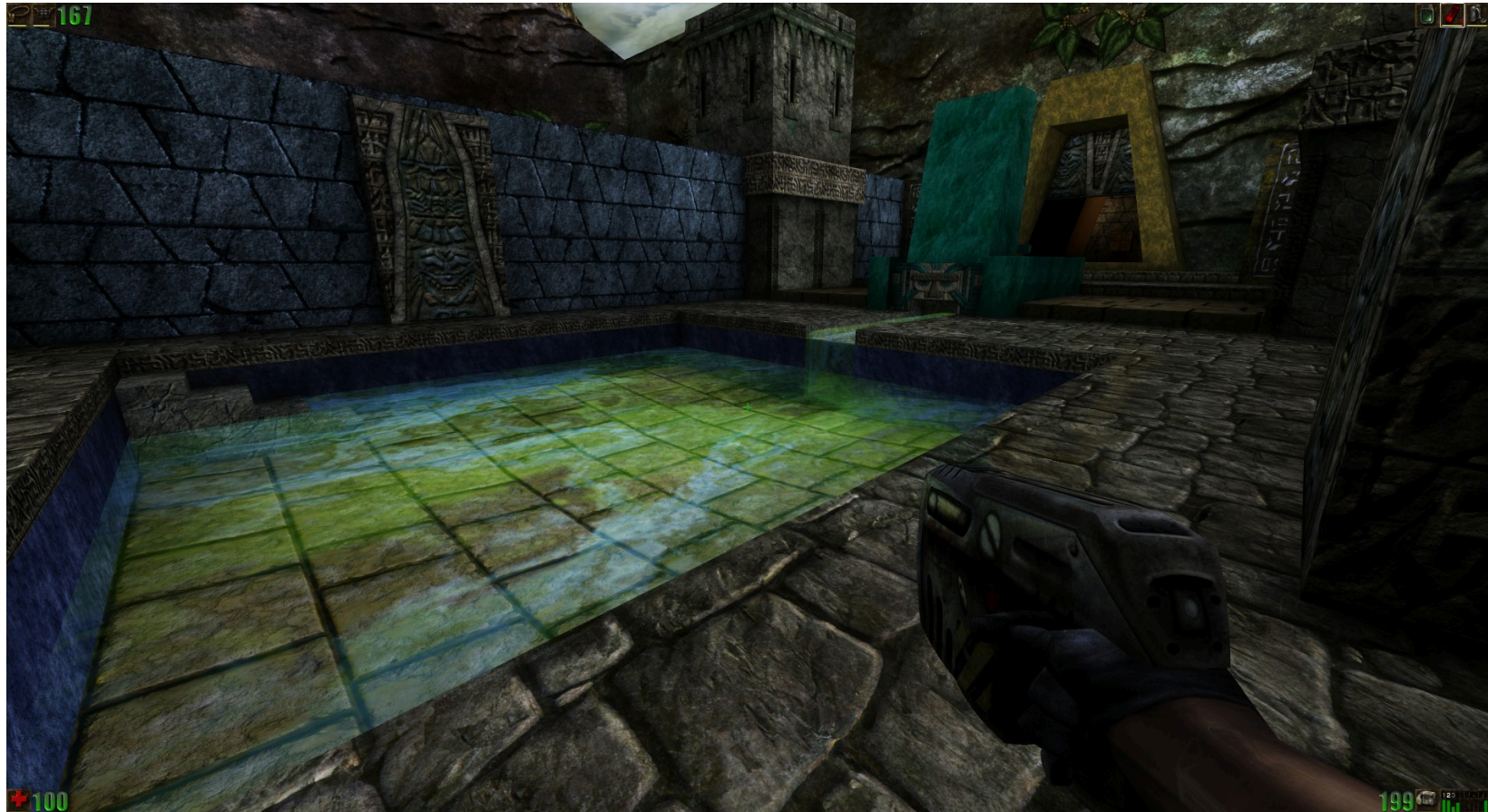


Figure 5: Unreal (1998)

Entwicklung der Computergrafik



Figure 6: Microsoft Flight Simulator 2024

Historischer Überblick

- Zeichenanzeigen (1960-heute)
- Vektordarstellung (1963-1980er Jahre)
- 2D-Bitmap-Rasteranzeigen für PCs und Workstations
- 3D-Grafik-Workstations
- Grafikkarten und GPUs

3D Computergrafik in Videospielen

- 1952: erstes grafisches Videospiel OXO von Alexander Douglas
- 1958: Tennis for two, erstes interaktives Videospiel
- 1961: Spacewar!, entwickelt am MIT
- 1972: Gründung von ATARI, Entwicklung von PONG
- 1978: Space Invaders, erstes Spiel in Farbe
- 1979: Asteroids > 50.000 verkaufte Exemplare
- 1994: Veröffentlichung von Sega Saturn und Sony Playstation
- 1996: Veröffentlichung von Nintendo 64
- Hardware ermöglichte erstmals 3D-Grafikdarstellung

3D Computergrafik in Filmen

- Pixars erster Kurzfilm: Die Abenteuer von Andre und Wally.B (1984)
- Toy Story als erster vollständig 3D-Animierter Film (1995)
- Vorgänger: Tin Toy (1988)
- Animationsfilme mit Blender:
 - Elephant's Dream
 - Big Buck Bunny

Neue Formen digitaler Medien

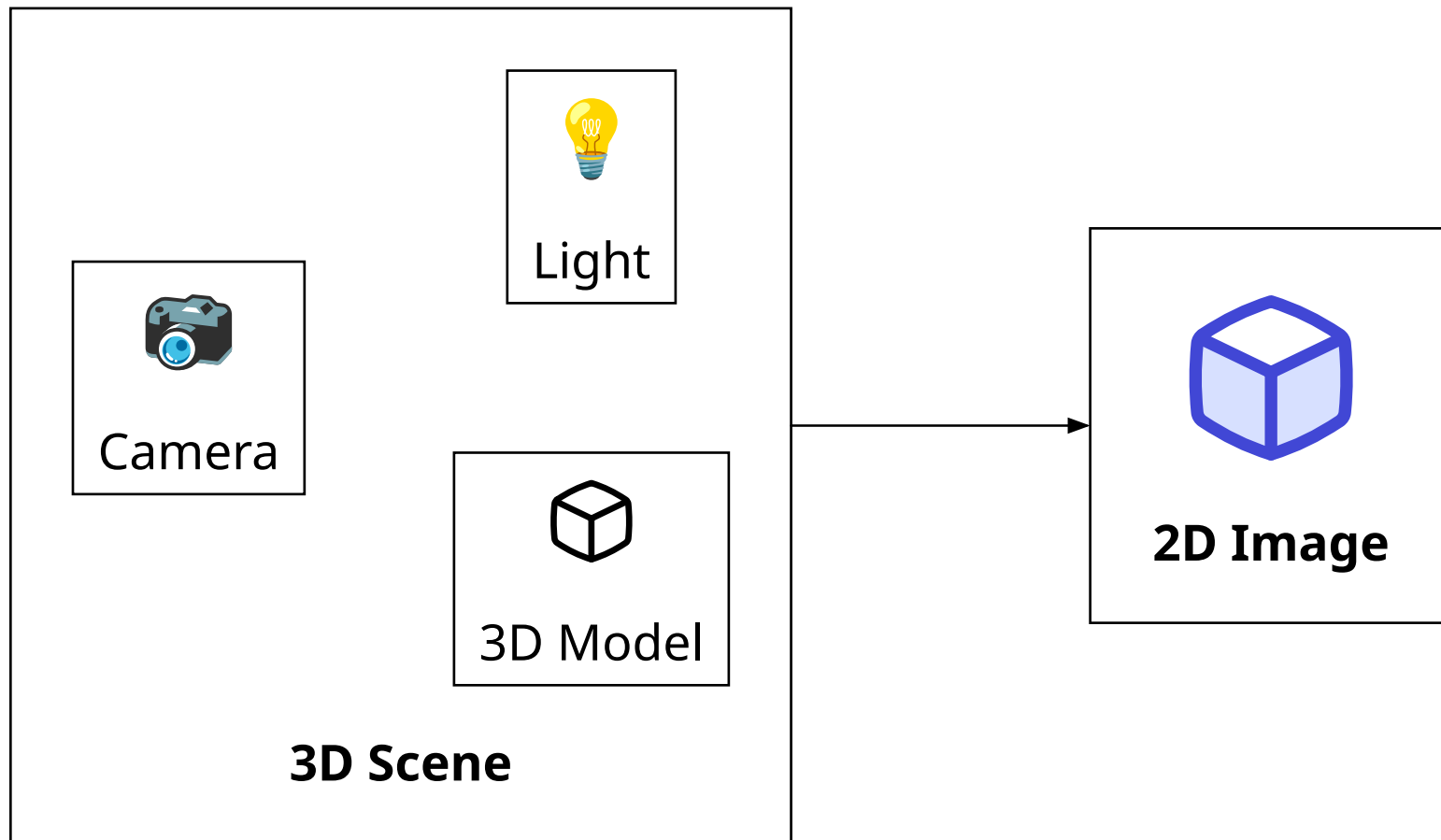
- Virtuelle Realität (fully immersive VR)
- CAVE, HMDs
- Semi-immersive VR
- Augmented Reality

Hardwareentwicklung

- **Spieleplattformen:**
 - Xbox Series: 52 CUs@1,825 GHz mit 12,15 TeraFlops
 - Playstation 5: AMD RDNA2 36 CUs @ 2,23 GHz mit 10,28 TeraFlops
- **Top Grafikkarten:**
 - Benchmarks ermöglichen Ranking von GPUs
 - momentan: GeForce RTX 5090
- **Advances in transistor counts ... (CPUs and GPUs)**
 - AMD Ryzen (2017) has 4.8 billion transistors
 - GeForce GTX 1080 (single core) has 7.2 billion transistors
 - Geforce GTX 2080 Ti (dual core) has 18.6 billion transistors
 - Radeon Fury has 8.9 billion, Radeon Pro Duo (dual core) has 17.8 billion transistors

Bestandteile der 3D Computergrafik

- **Modeling** = Repräsentation von 3D Objekten
- **Rendering** = Erstellung von 2D Bildern aus 3D Modellen
- **Animation** = Simulation von Zustandsänderungen über die Zeit



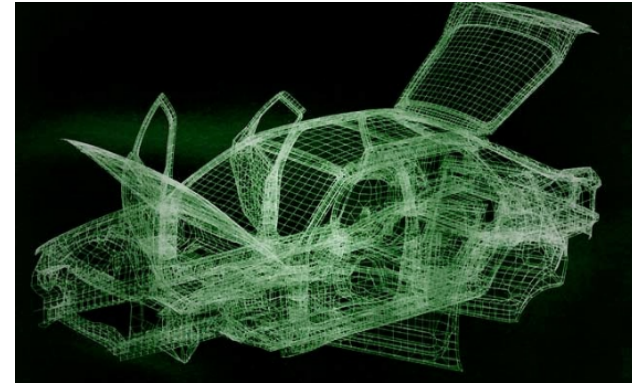
Rendering

Computergrafik Paradigmen



Sample-Based Graphics

- Diskrete *Samples* werden für visuelle Informationen benutzt
- Pixels werden durch digitalisierung von Bildern erstellt
- Beispiele: Photoshop, GIMP, Krita



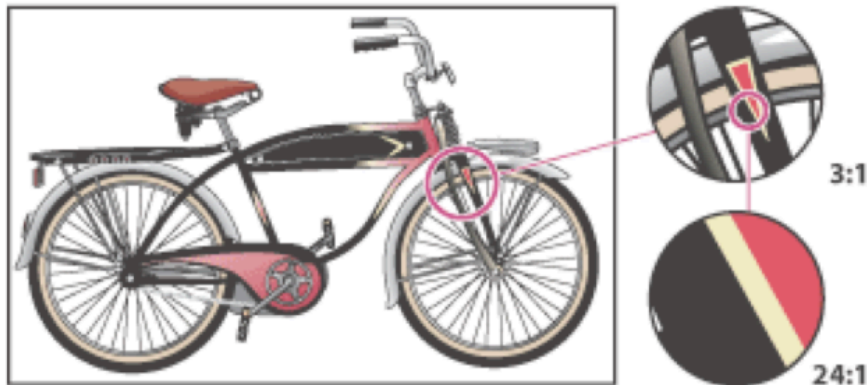
Geometry-Based Graphics

- Geometrisches Modell wird zusammen mit Darstellungs-Attributen erstellt
- Modell wird gesampled zur Darstellung (rendering)
- Für 2D: Illustrator, Inkscape
- Für 3D: CAD, Blender, Maya

Sample-Based vs. Geometry-Based

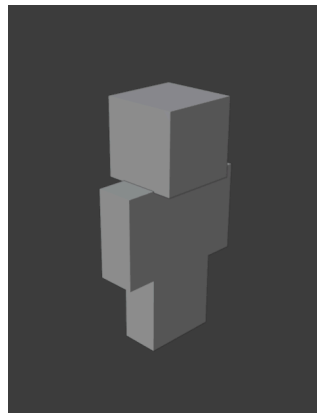


Für **Sample-Based** Grafiken ist der originale Datensatz selbst ein Sample (z.B. ein Foto). Es kann keine bessere Auflösung als die des Originals erlangt werden.



Bei **Geometry-Based** Grafiken hat der Computer ein mathematisches bzw. geometrisches Modell als Basis. Dieses Modell erlaubt Skalierung und Zoom ohne Qualitätsverlust.

Kombinieren der Paradigmen



+



=



- Geometry-Based und Sample-Based Grafiken werden beim Rendering oft kombiniert
- 3D Modell ist Geometry-Based
- Texturen des Modells sind Sample-Based Grafiken
- Verbesserte Qualität und Performance des Endprodukts

Sampling für den Monitor

Sampling zur finalen Darstellung

Hinweis 2

Immer wenn ein Bild auf einem Monitor dargestellt wird, muss Sampling durchgeführt werden. Dabei ist es egal ob die Grafik, die wir darstellen wollen, Geometry-Based oder Sample-Based ist. Die Grafik muss in die Pixel des Monitors *gepackt* werden. Geometry-Based Grafiken erlauben hier nur unlimitiertes Skalieren.

Rendering

Linien-Rendering

Linien-Rendering

Bresenham's Linien Generierungs Algorithmus

Es seien zwei Punkte gegeben: $A(x_0, y_0)$ und $B(x_1, y_1)$.

Ziel ist es, alle dazwischen liegenden Punkte zu finden, die man benötigt, um eine Linie auf einem Pixel Display zu zeichnen. Jeder Pixel hat dabei Integer Koordinaten.

Beispiele:

Input: $A(0, 0)$, $B(4, 4)$

Output: $(0, 0)$, $(1, 1)$, $(2, 2)$, $(3, 3)$, $(4, 4)$

Input: $A(0, 0)$, $B(4, 2)$

Output: $(0, 0)$, $(1, 0)$, $(2, 1)$, $(3, 1)$, $(4, 2)$

Linien-Rendering

Bresenham's Linien Generierungs Algorithmus

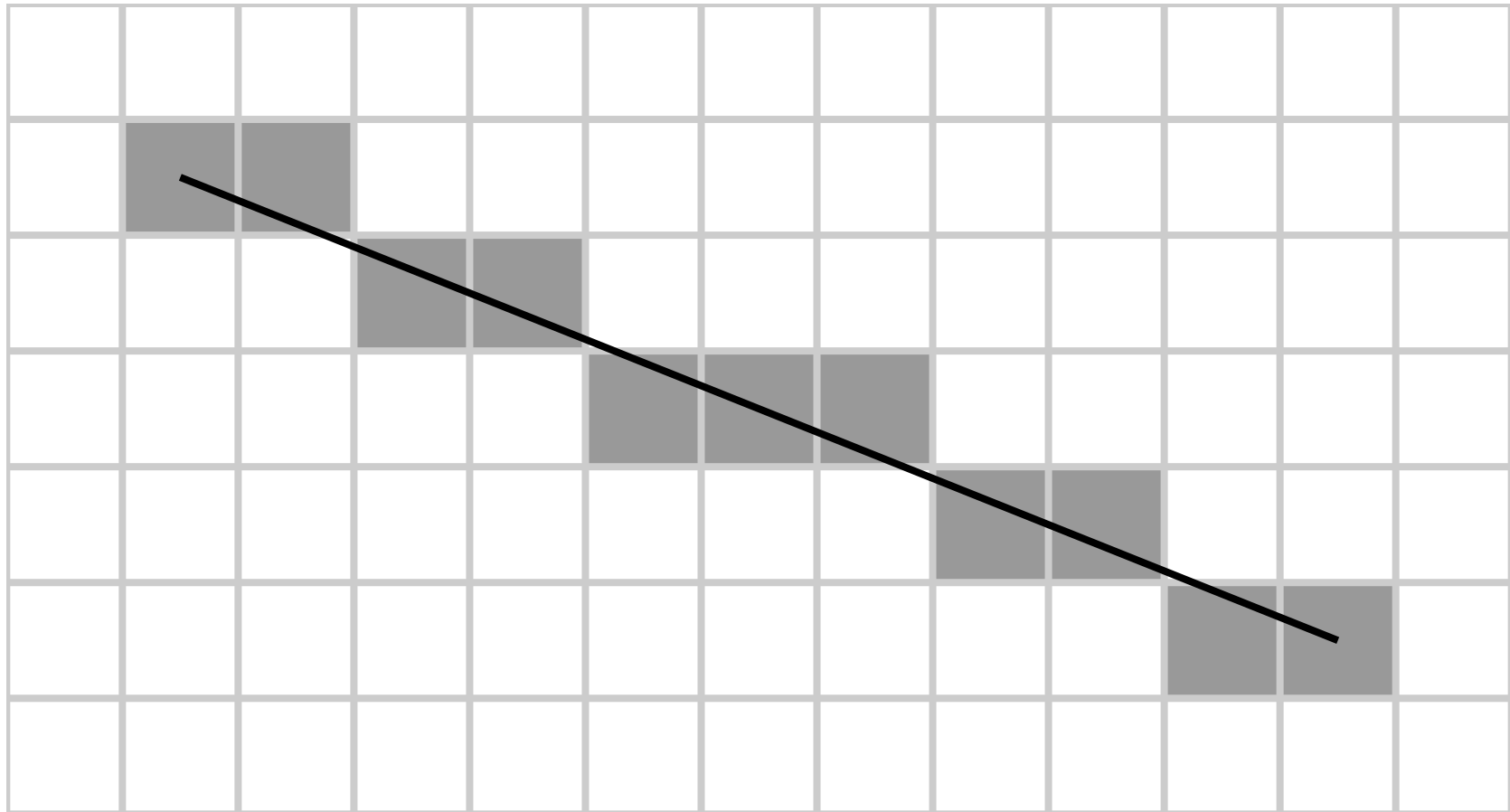


Figure 12: Ergebnis des Bresenham's Linien Algorithmus. Der Anfang der Linie ist bei $A(1, 1)$ oben links und das Ende ist bei $B(11, 5)$

Rendering

Polygon-Rendering

Polygon-Rendering

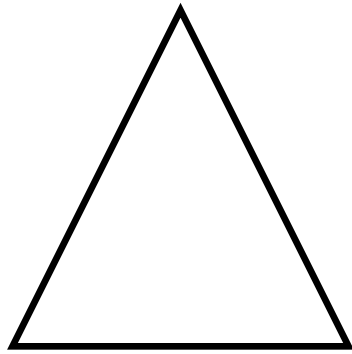


Figure 13: Ungefülltes Polygon

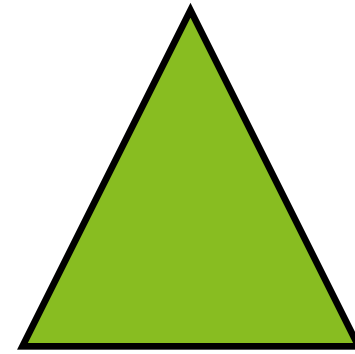


Figure 14: Gefülltes Polygon

Polygon-Rendering



Figure 15: Wireframe Modell einer Cobra in dem ZX Spectrum Port von Elite. Man beachte, dass Linien auf der Rückseite der Cobra nicht angezeigt werden. Es kommt **Hidden-Line Removal** zum Einsatz.

Polygon-Rendering



Figure 16: Gefüllte Polygone einer Cobra in Elite. Die Polygone besitzen hier kein Shading.

Polygon-Rendering

Ungefüllte Polygone

- Einfache Sequenz von gerenderten Linien
- Bresenham's Linien Generierungs Algorithmus
- Nutzung in Wireframe Rendering

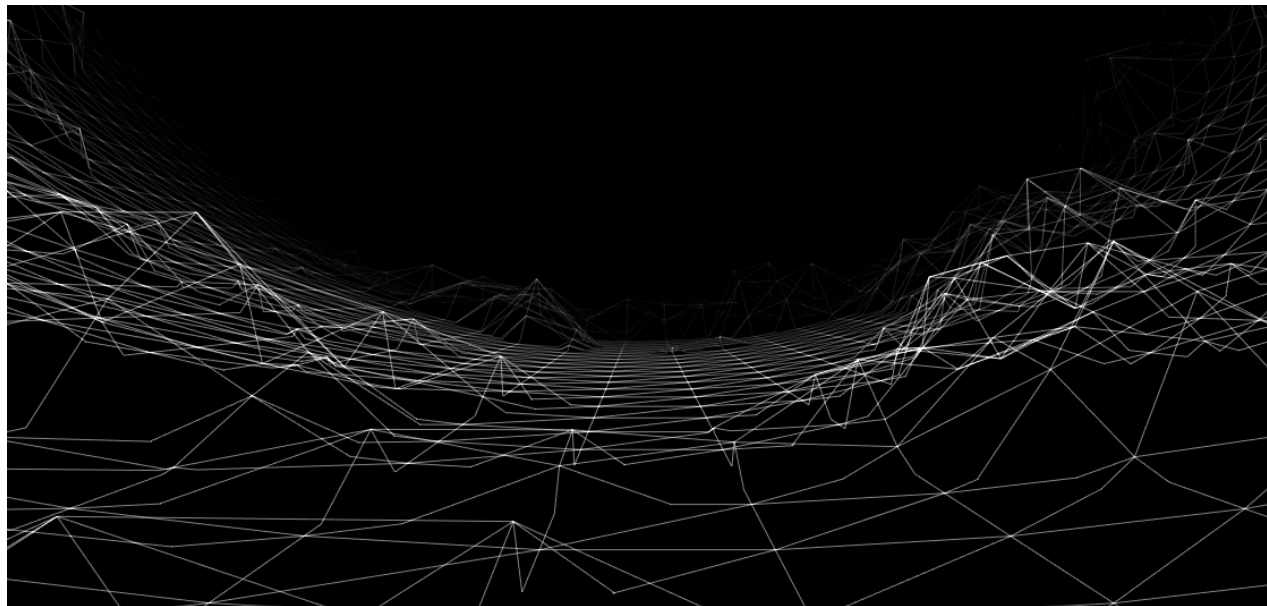
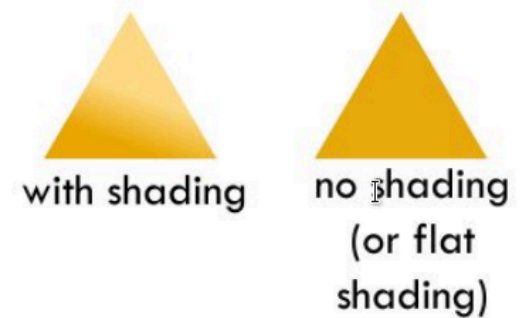
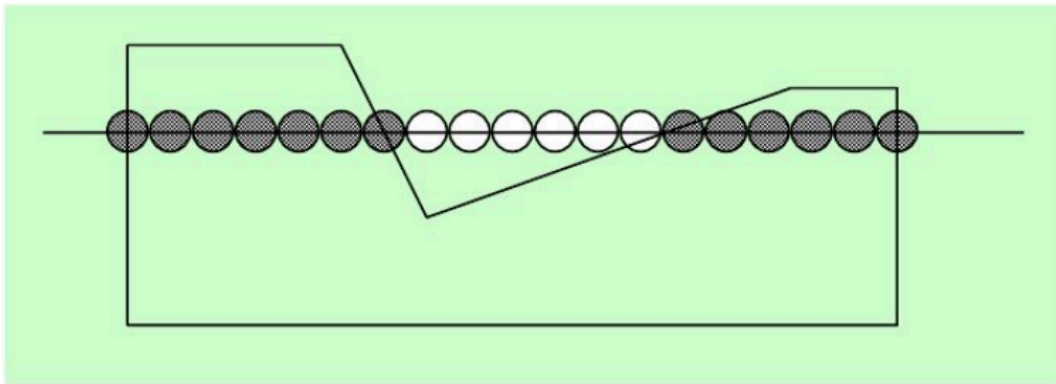


Figure 17: Landschaft als Wireframe gerendert.

Polygon-Rendering

Gefüllte Polygone

- Füllen des Polygons mit jeweils einer Scanlinie



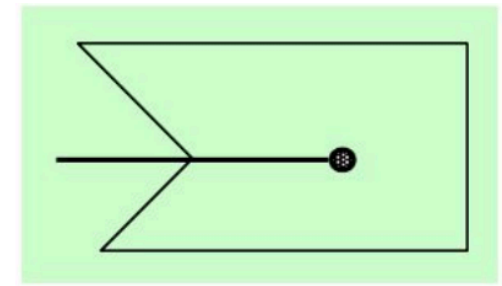
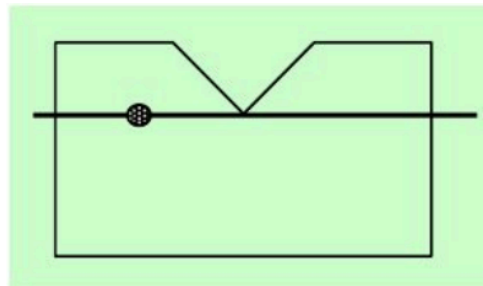
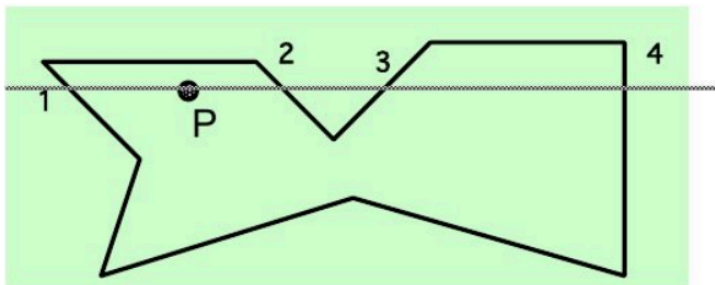
- Bestimmen, welcher Pixel auf der Scanlinie in dem Polygon liegt
- Überprüfung jedes Pixels wäre unperformant
- **Schlüsselidee:** Nicht jeder Pixel wird auf "*inside-ness*" überprüft. Es werden nur Pixel gesucht, an denen Änderungen der "*inside-ness*" auftreten.

Polygon-Rendering

Gefüllte Polygone

Inside-Test

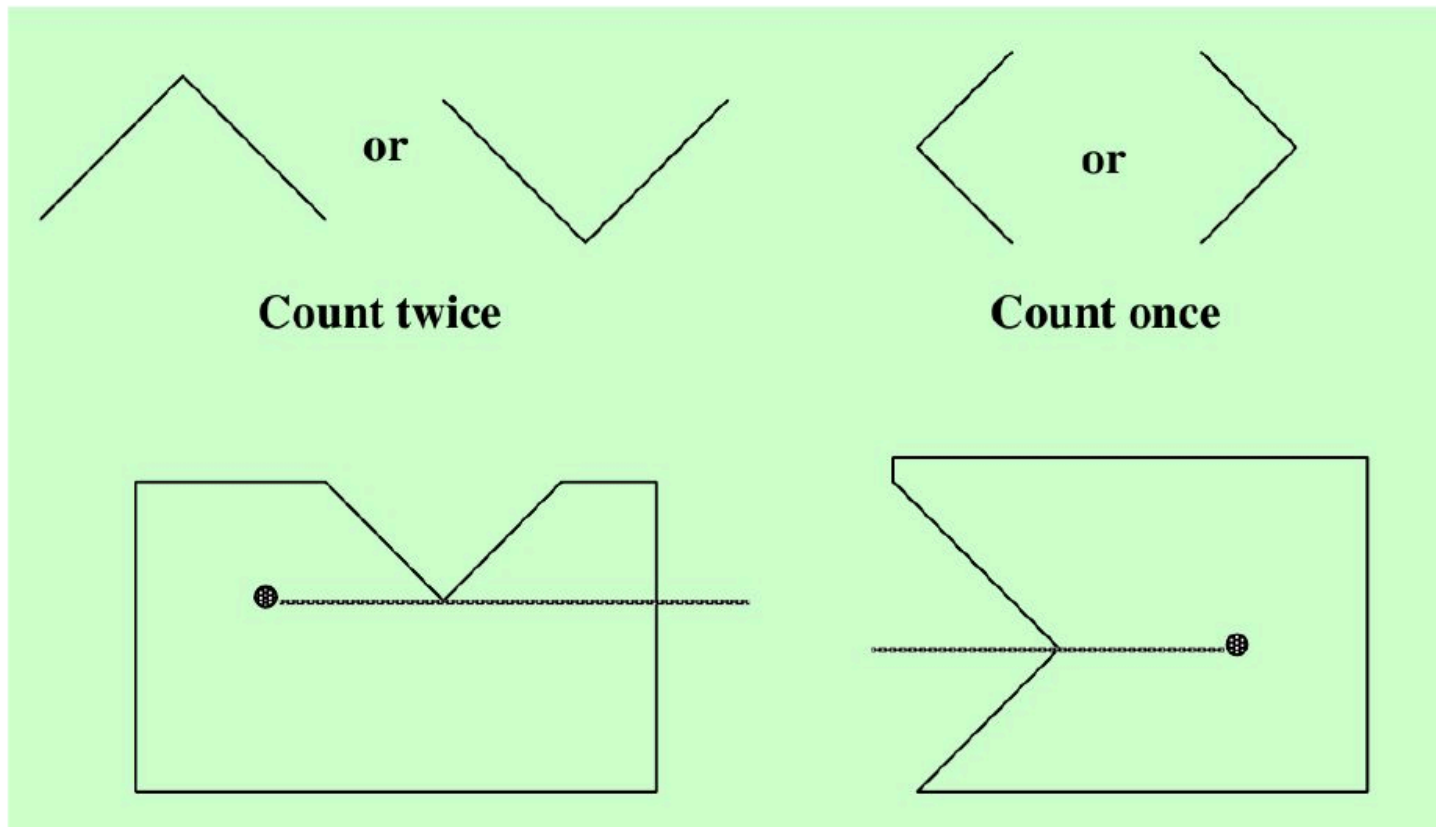
Ein Punkt P befindet sich nur dann innerhalb eines Polygons, wenn eine Scanlinie die Polygonkanten eine **ungerade** Anzahl an Malen schneidet und sich von P in beide Richtungen bewegt.



Polygon-Rendering

Gefüllte Polygone

- Max-Min-Test: 2x, wenn lokales Maximum/Minimum, sonst 1x



Polygon-Rendering

Gefüllte Polygone

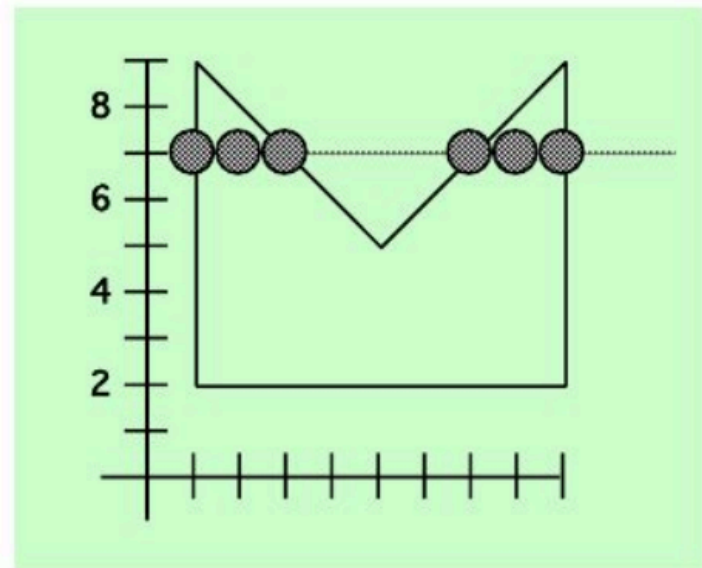
Scan-Line Algorithmus

Für jede Scanline:

1. Finden Sie die Schnittpunkte der Scanlinie mit allen Kanten des Polygons.
2. Sortieren Sie die Schnittpunkte nach zunehmender x-Koordinate.
3. Füllen Sie alle Pixel zwischen den Schnittpunktpaaren aus.

For scan-line number 7 the sorted list of x-coordinates is (1,3,7,9)

Therefore fill pixels with x-coordinates 1-3 and 7-9.

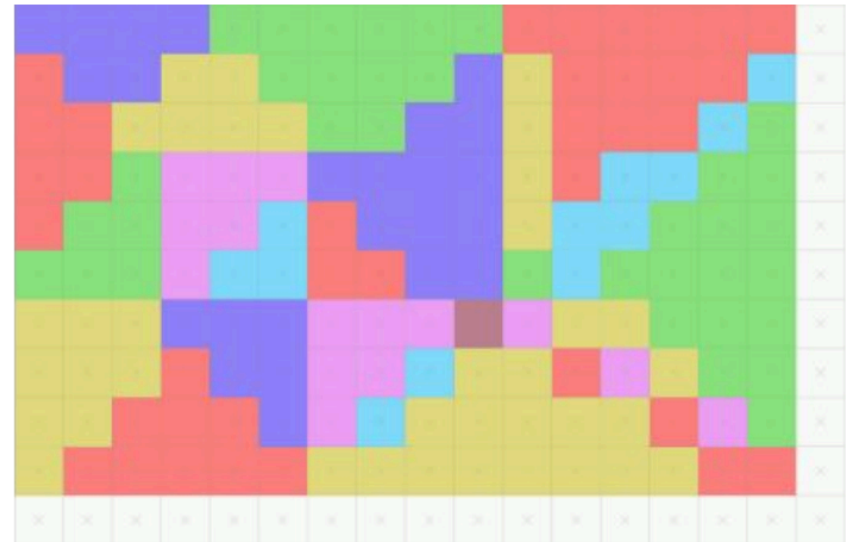
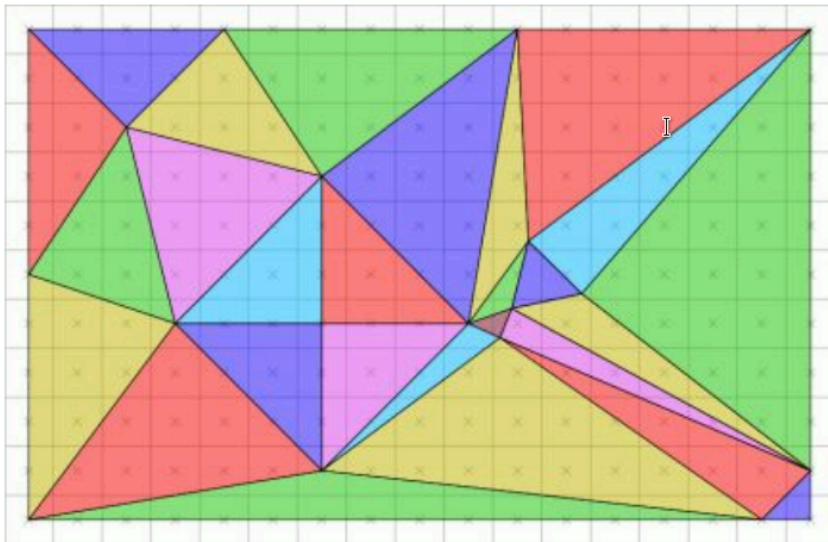
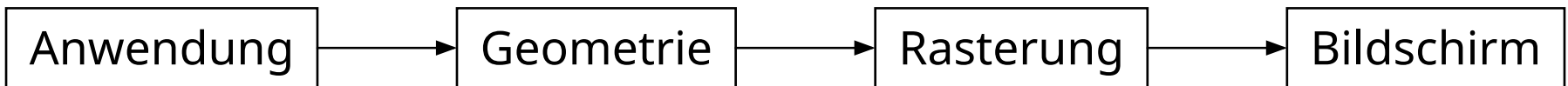


Rendering

Rendering Pipeline

Klassische Rendering-Pipeline

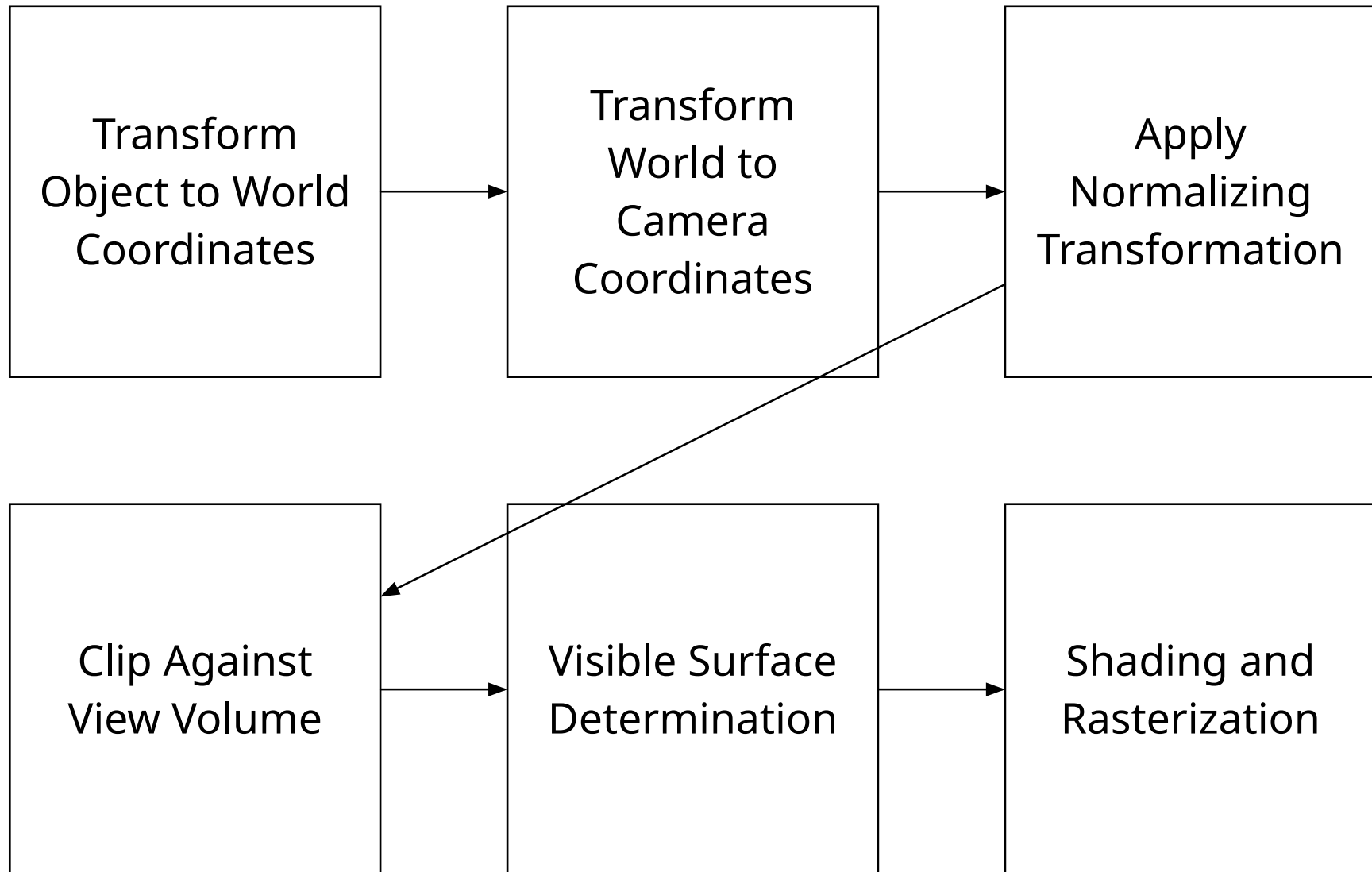
- Pipeline = Arbeitsschritte zur Darstellung von 2D Grafiken von 3D Objekten



Klassische Rendering-Pipeline

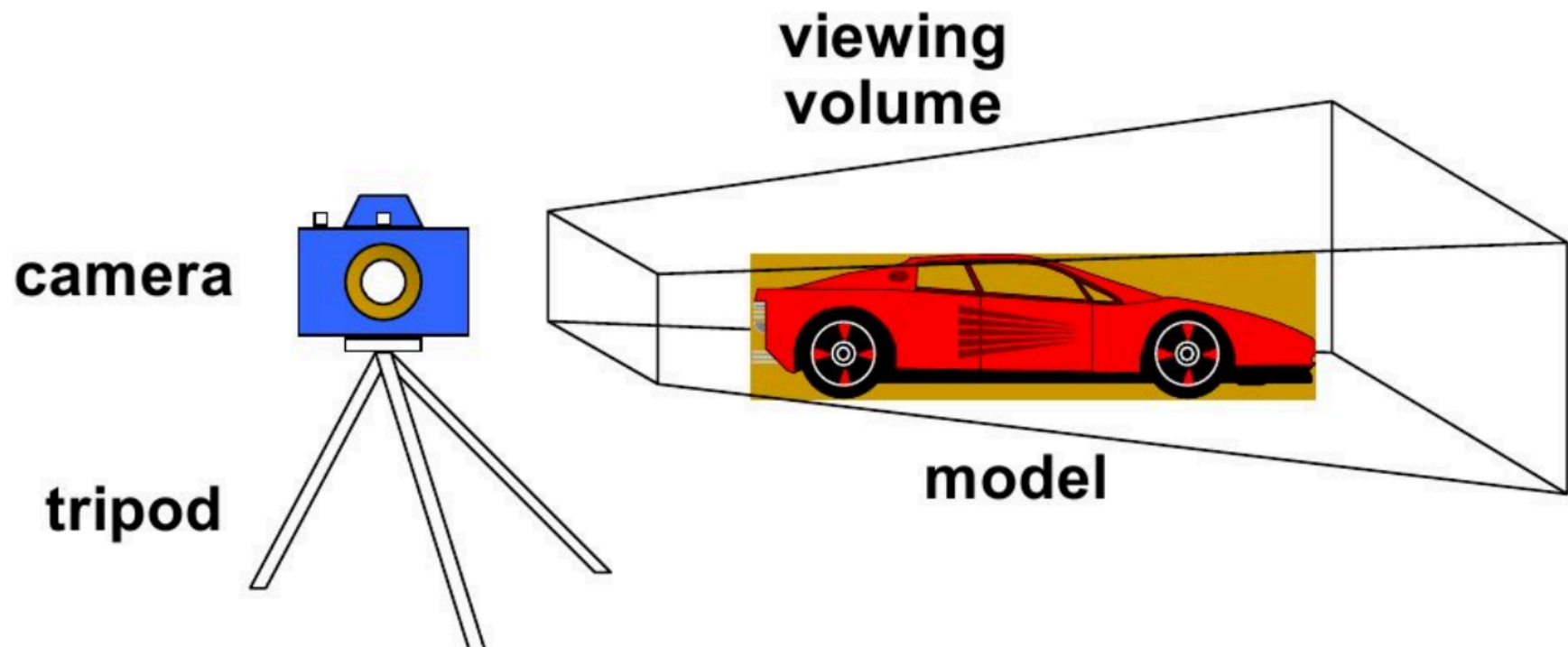
- Umwandeln der 3D-Welt in (normalisierte) Kamerakoordinaten
- Ausschnitt & Bestimmung der sichtbaren Oberflächen
 - Berechnen der Menge der vom Betrachtungspunkt aus sichtbaren Oberflächen
- Rasterung (Scan-Konvertierung)
 - Beleuchtung und Schattierung (lokale, direkte Beleuchtungsmodelle): Rendertiefe, Beleuchtungseffekte, Materialeigenschaften

Klassische Rendering-Pipeline



Klassische Rendering-Pipeline

- 3D ist wie das Aufnehmen eines Fotos (... vieler Fotos)

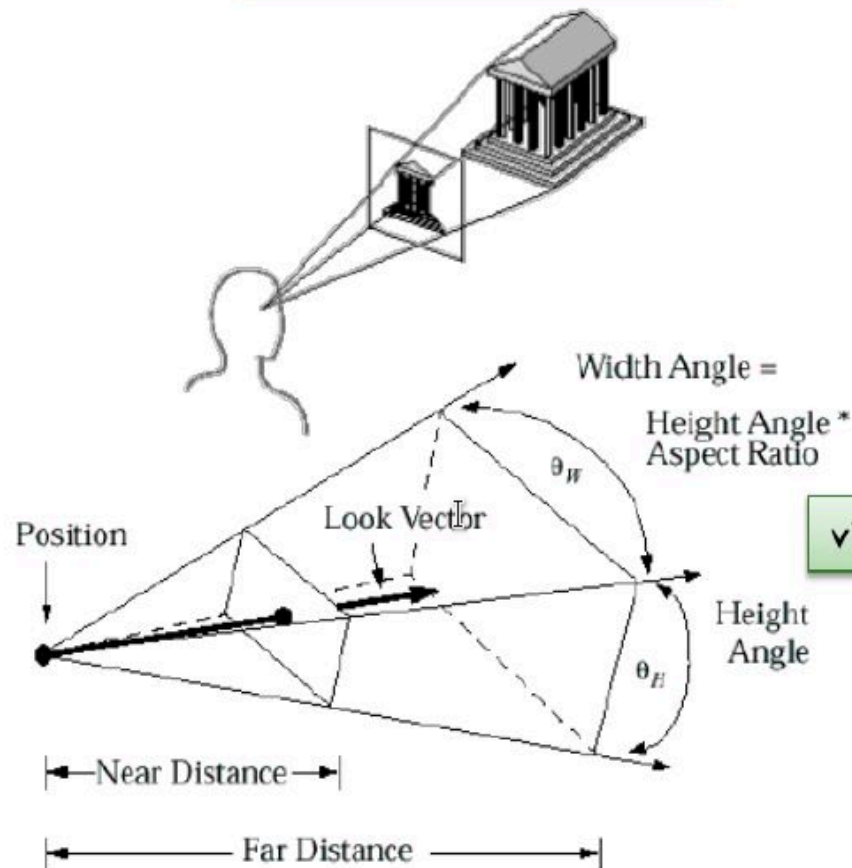


Synthetische Kamera

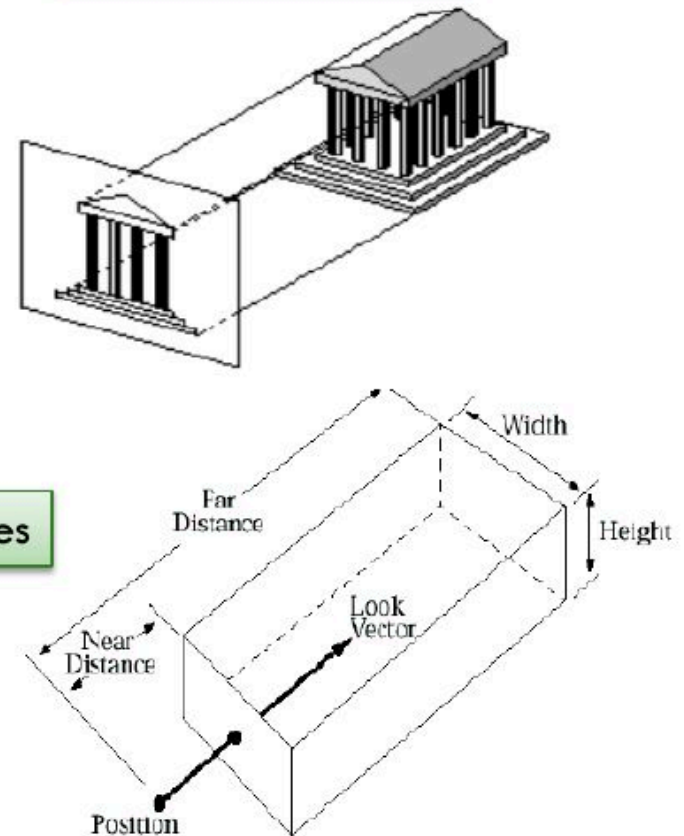
- Kamera/Viewpoint Location
- Kamera/Viewpoint Direction
- Kamera/Viewpoint Orientation
- Kamera/Viewpoint Lens (View Volume)
 - Breite/Höhe der Lens
 - Front/Back Clipping Planes
- Parallele oder perspektivische Projektion

Parallele und perspektivische Projektion

Perspective Projection



Parallel Projection



view volumes

Rendering

Canonical View Volume

Canonical View Volume

- Ein, oft als Box, definierter Bereich, der alles abdeckt, was die Kamera sieht
- Alles, was sich in dem Canonical View Volume befindet wird gerendert.
- Geometrie aus der Kamerasicht wird auf einen festen und normalisierten Bereich überführt

Beispiel: Vulkan

X from $(-1, 1)$

Y from $(-1, 1)$

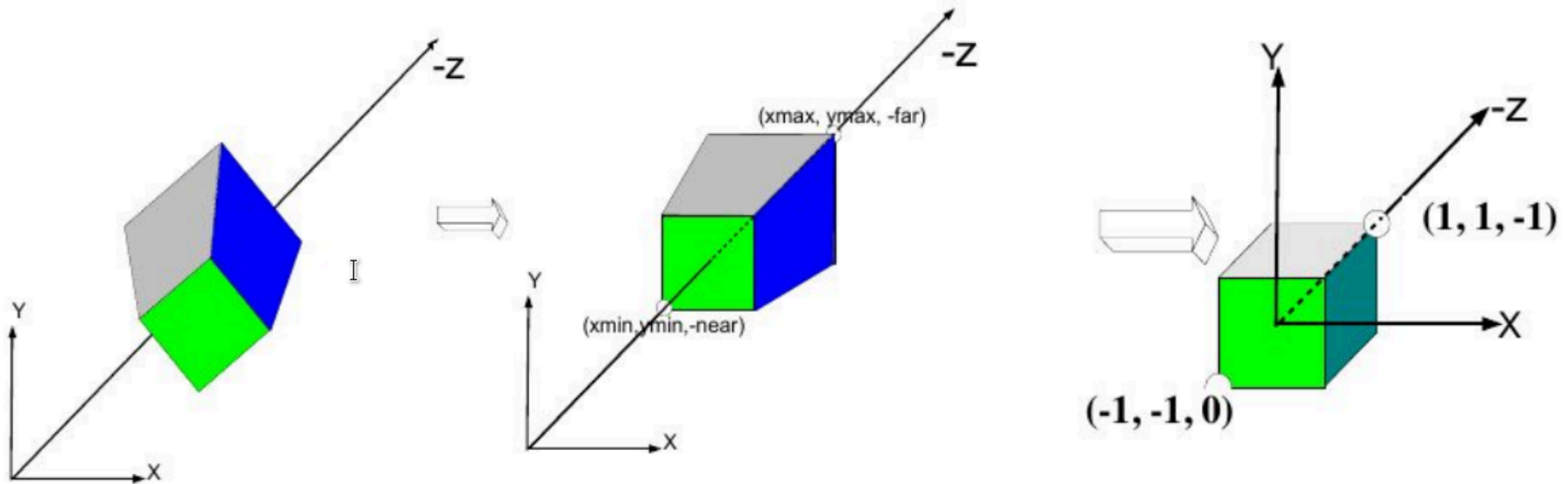
Z from $(0, 1)$

Z normalisiert Tiefenwerte. Einfacheres Depth-Testing

Canonical View Volume

Welt verzerren, bis das Betrachtungsvolumen in Welt in ein paralleles kanonisches Betrachtungsvolumen passt

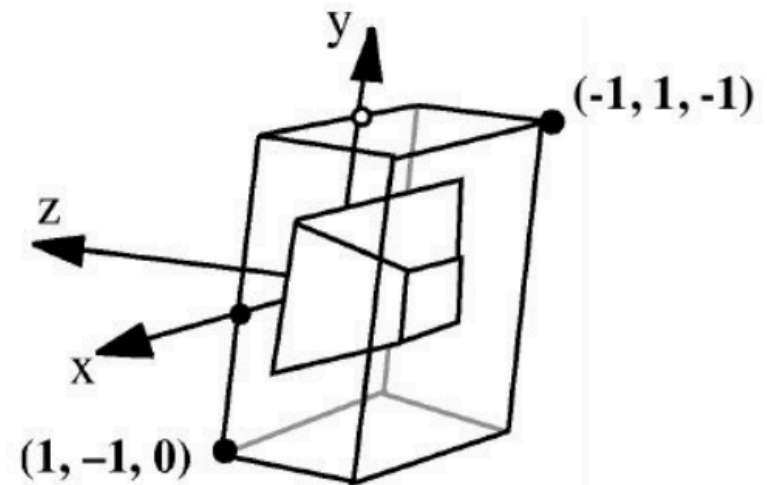
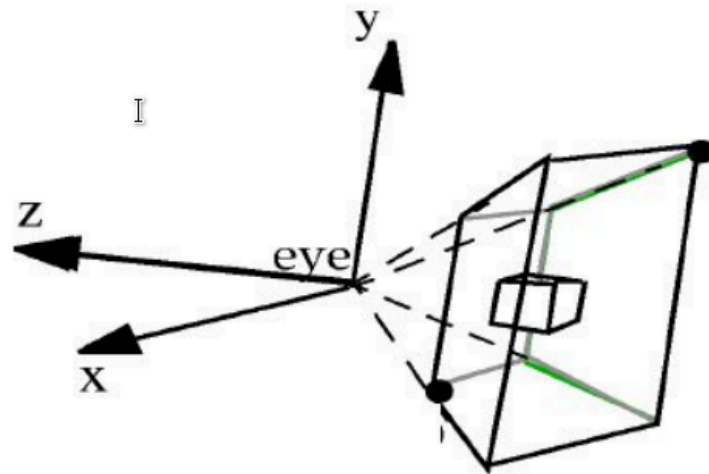
- vereinfacht Beschneiden/Entfernen verdeckter Oberflächen/
Schattierungen
- durch die Multiplikation mit einer 4 x 4-Transformationsmatrix



Canonical View Volume

Die 3D zu 2D-Projektion (in die "Projektionsebene") ist jetzt einfach:

- Ignorieren Sie einfach den z-Wert!
- In Wirklichkeit behalten wir den z-Wert für die Entfernung verdeckter Oberflächen (Z-Buffer) und Schattierungseffekte bei.



Canonical View Volume

Viewing Frustum (Sichtkegel)

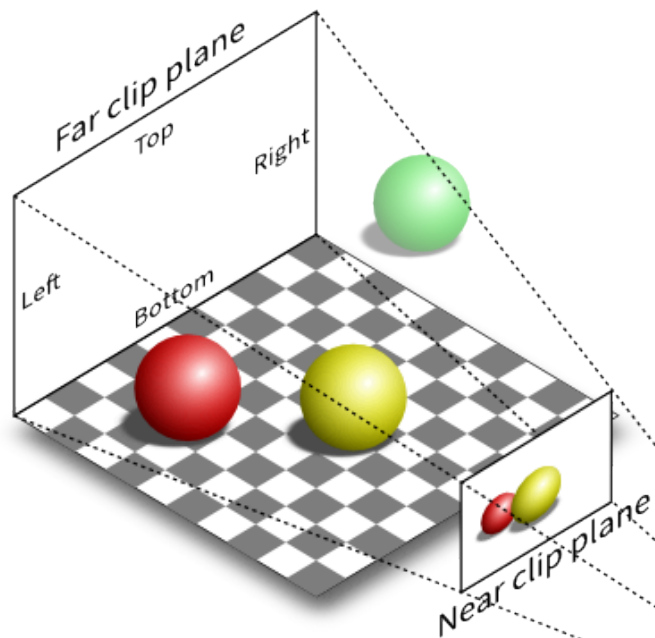
- Canonical View Volume ist nah mit dem Viewing Frustum verwandt
- Viewing Frustum gibt die Region im Raum an, die auf dem Bildschirm erscheinen wird

Bestandteile:

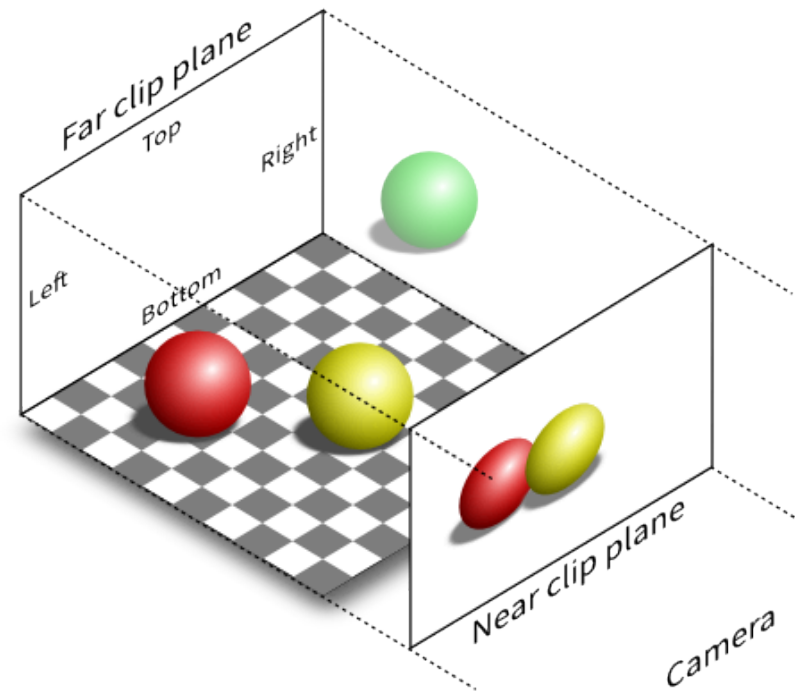
- **Near Plane:** Die nahe Grenze zur Kamera
- **Far Plane:** Die ferne Grenze zur Kamera
- **Field of View:** Der Winkel in vertikale Richtung
- **Aspect Ratio:** Das Seitenverhältnis zwischen Breit und Höhe

Canonical View Volume

Viewing Frustum (Sichtkegel)



Perspective projection (P)

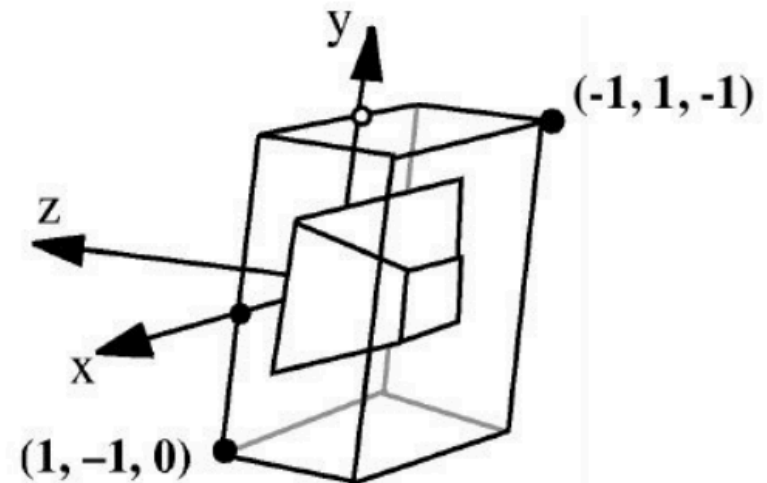
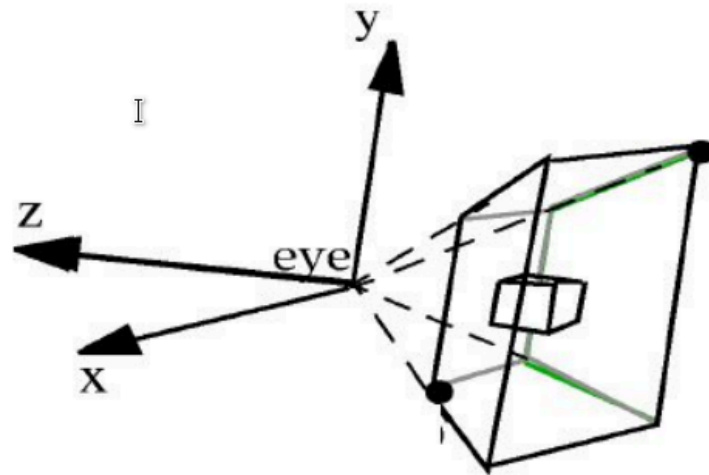


Orthographic projection (O)

Canonical View Volume

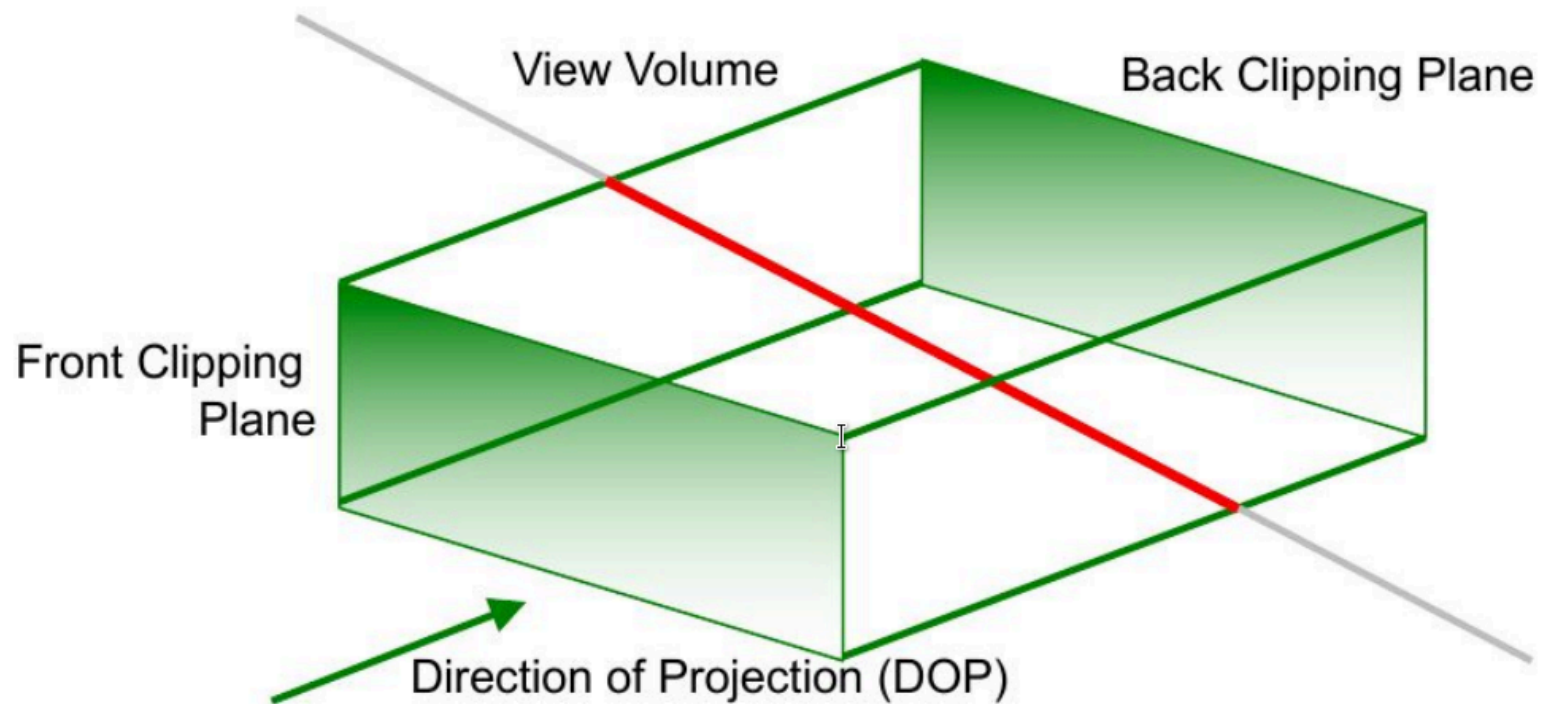
Viewing Frustum (Sichtkegel)

- Der Viewing Frustum wird in das Canonical View Volume transformiert
- Nach der Transformation erfolgt Clipping und Rasterisierung



Clipping

- Polygone außerhalb der Ansichtsvolumen verwerfen
- Polygone gegen Ansichtsvolumen beschneiden
 - auch leicht im Canonical View Volume durchführbar
 - Einfach gegen eine Box zu clippen als gegen einen Sichtkegel



Visible Surface Determination

- **Objekt-Präzisionsmethoden**

- pro Polygon; Welt-/Kamerakoordinaten; Fließkommazahlen
- z.B. Backface Culling
 - vom Betrachter abgewandte Polygone verwerfen
- z.B. Occlusion Culling
 - Polygone verwerfen, die durch andere Polygone verdeckt sind
 - z.B. Portal-Rendering - Szene in Zellen/Sektoren (Räume) und Portale (Türen) unterteilen

- **Bild-Präzisionsverfahren**

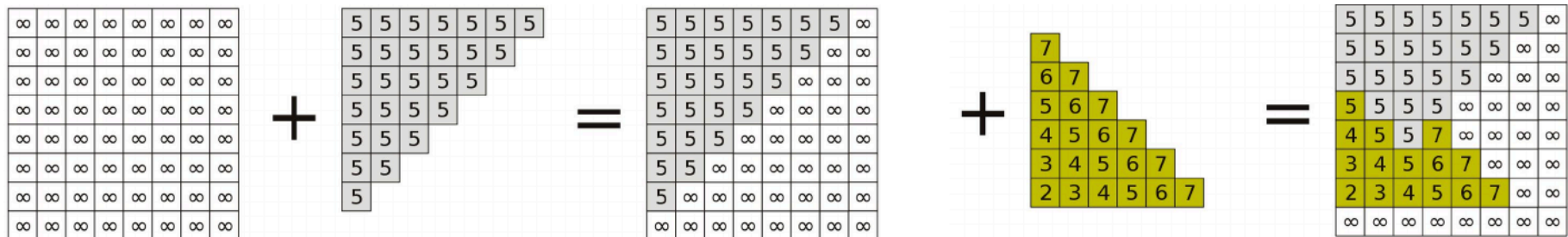
- pro Pixel; Bildschirmkoordinaten; ganzzahlige Werte
- z.B. Z-Buffer
- z.B. Occlusion Culling durch Off-Screen-Rendering

Rendering

Z-Buffer

Z-Buffer

- Buffer mit Pixel-Informationen
 - Bild-Buffer, mit Pixelfarbe
 - Z-Buffer, mit Pixeltiefeninformationen
- Polygone werden in beliebiger Reihenfolge gerendert
- Alle Polygone werden gerendert
- Z-Buffer wird verwendet, um bei Überlappungen zu entscheiden welches Polygon Pixel *bekommt*
- Kein Sortieren notwendig



Z-Buffer

- sehr schnell
- In fast allen Grafikkarten implementiert
- kann aber kein Anti-Aliasing durchführen
 - erfordert die Kenntnis aller an einem bestimmten Pixel beteiligten Polygone
- Problem: **Z-Fighting**

Z-Buffer

Z-Fighting

- kann im Falle von komplanaren Polygonen auftreten (Abbildung)
- kann durch eine begrenzte Auflösung des Tiefenpuffers verursacht werden (z.B. 16 Bit)

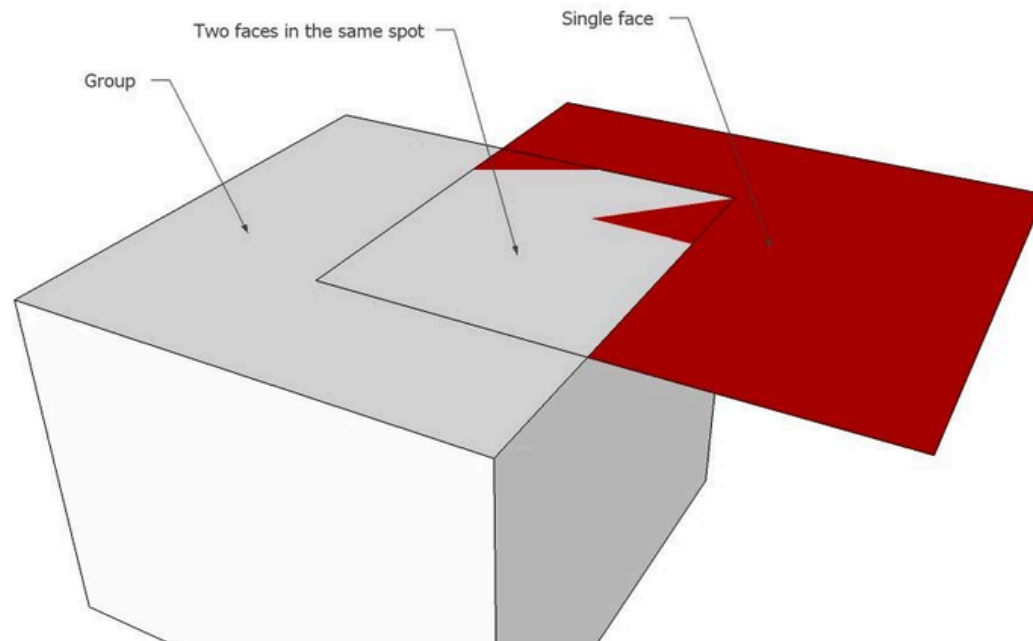


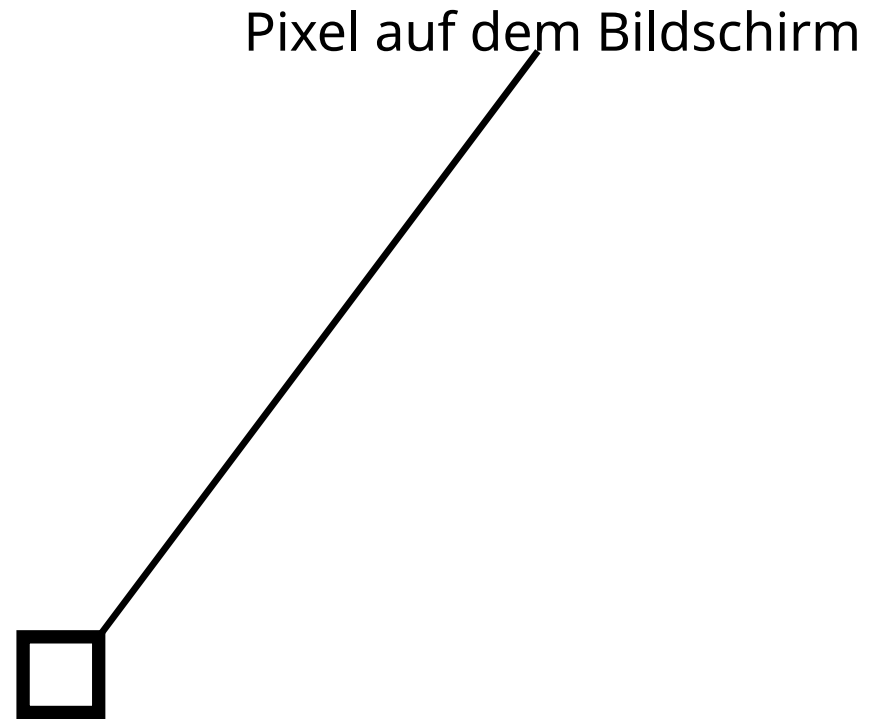
Figure 32: Z-Fighting zwischen einer *Plane* und einem *Cube*

Z-Buffer

Beispiel

Aktuelle Z-Depth: 1.0

Je kleiner die Z-Depth,
desto näher ist das Objekt
an der Kamera



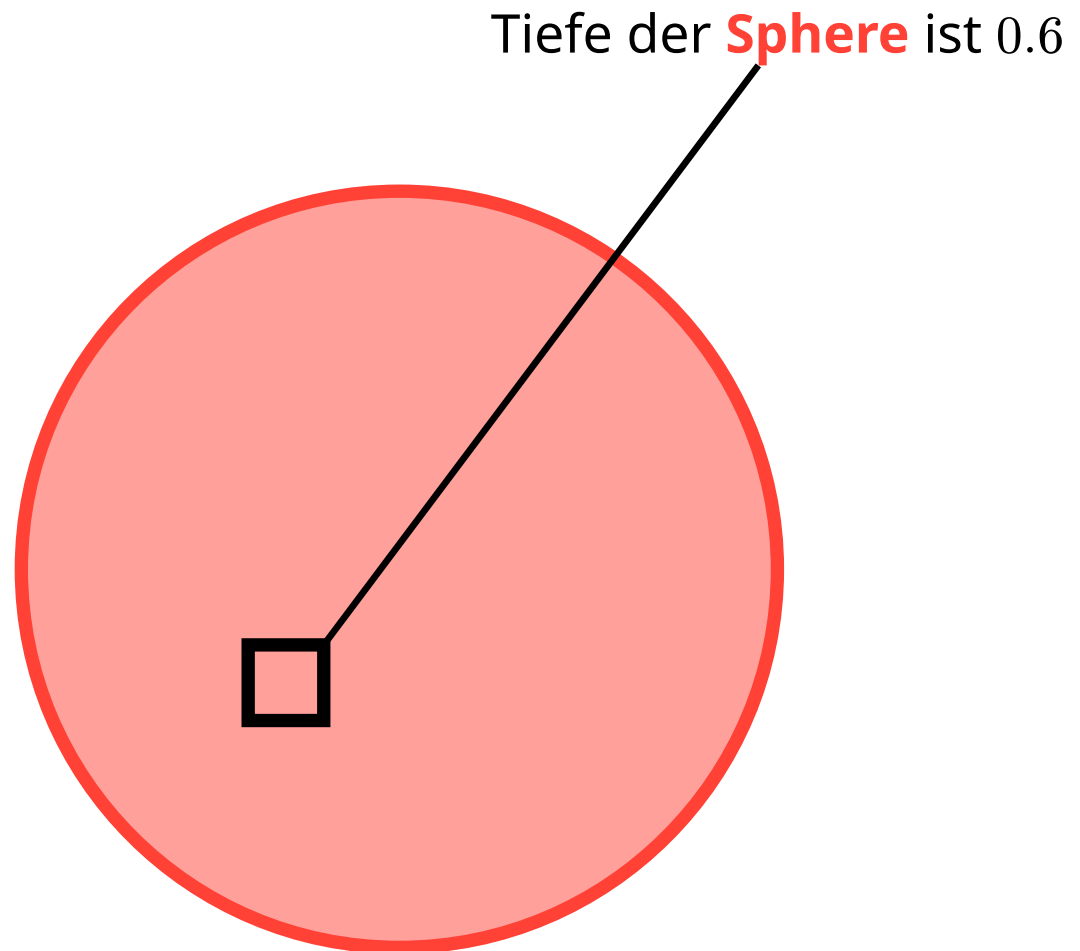
Z-Buffer

Beispiel

Aktuelle Z-Depth: 1.0

$$1.0 > 0.6$$

Neue Z-Depth: 0.6



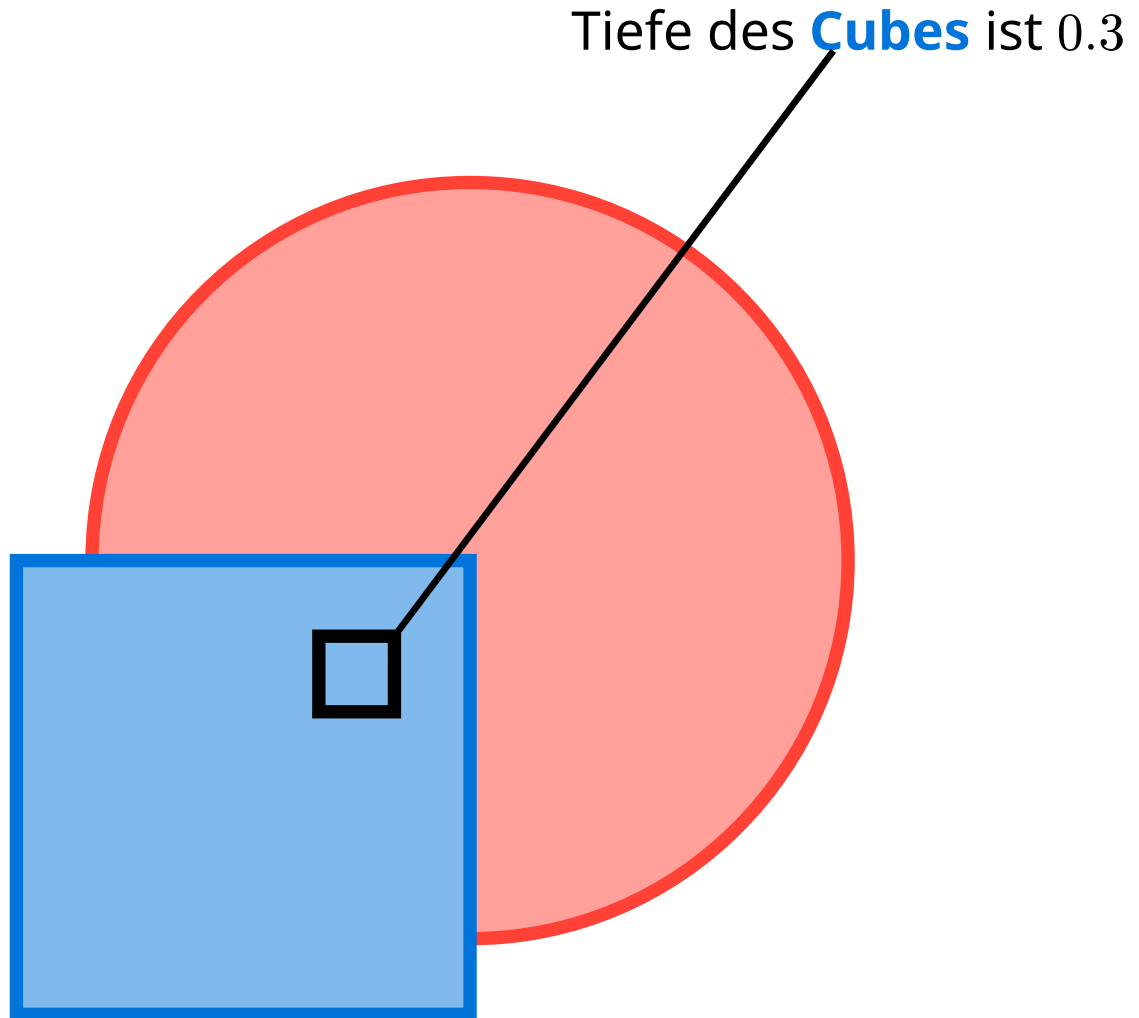
Z-Buffer

Beispiel

Aktuelle Z-Depth: 0.6

$$0.6 > 0.3$$

Neue Z-Depth: 0.3



Z-Buffer

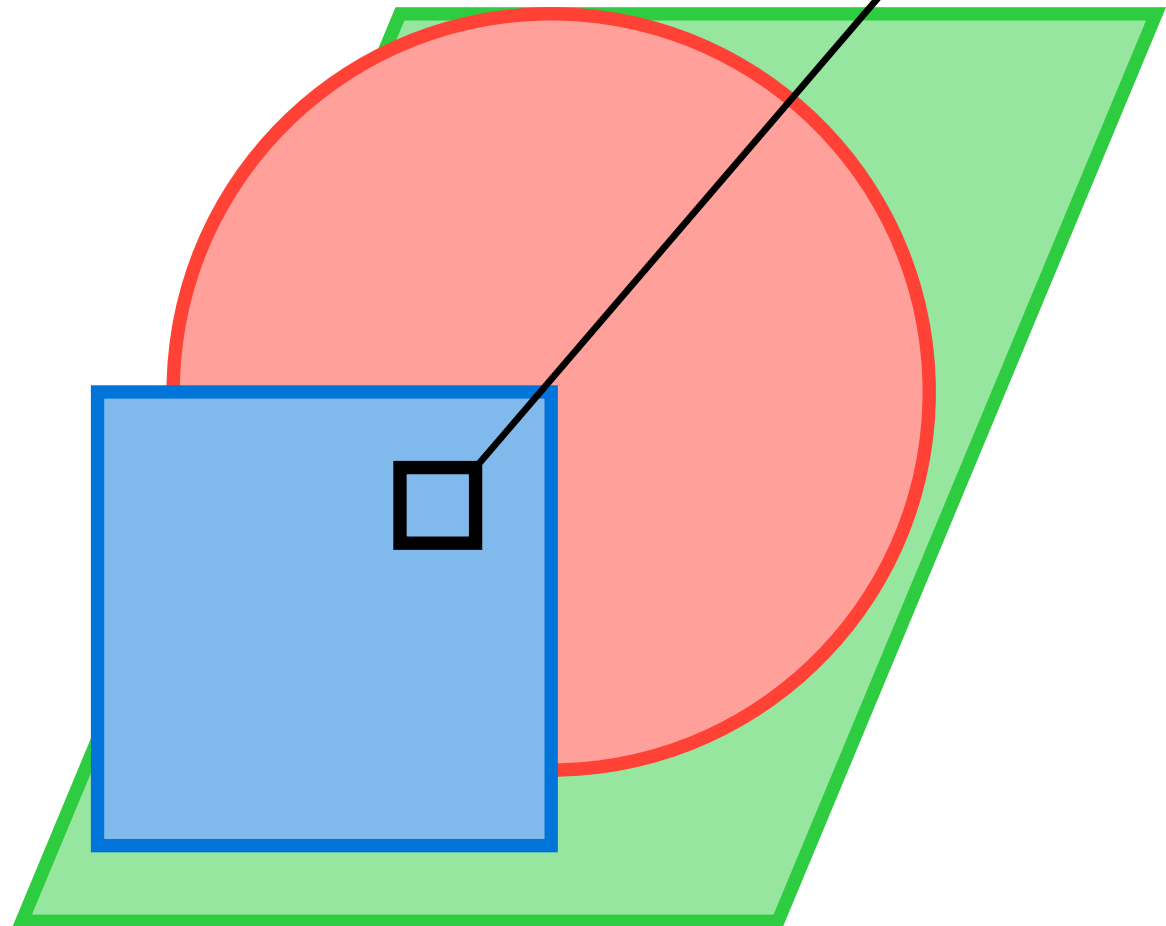
Beispiel

Tiefe der **Plane** ist 0.9

Aktuelle Z-Depth: 0.3

$$0.9 > 0.3$$

Neue Z-Depth: 0.3



Rendering

Anti-Aliasing

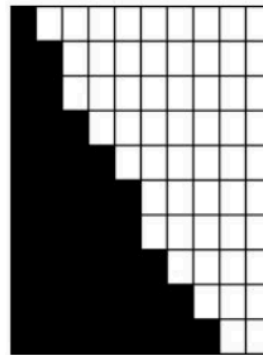
Anti-Aliasing

- Probleme mit der Rasterung
 - Darstellung einer Linie mit diskreten Pixelwerten ist die Abtastung einer
 - *Jaggies* sind eine Manifestation von Stichprobenfehlern und Informationsverlust kontinuierlichen Funktion
- Antialiasing-Idee (für Linienwiedergabe):
 - Rechteck (1 Pixel breit) statt mathematischer Linie
 - Pixelintensität abhängig von der Überlappung mit dem Einheitsrechteck:
 - wie viel Überlappung
 - wo Überschneidungen auftreten

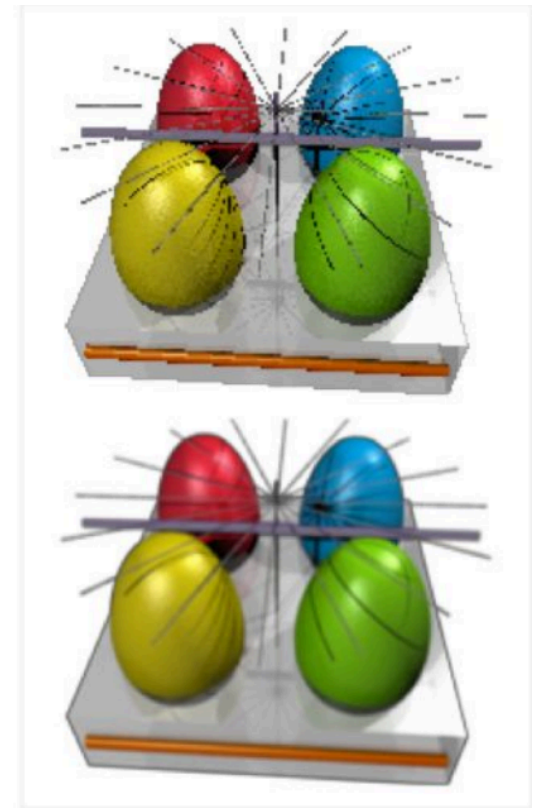
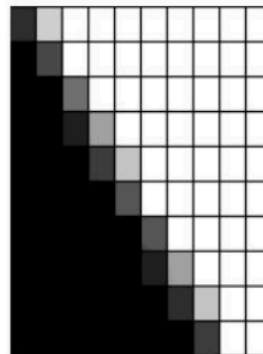
Anti-Aliasing

with jaggies

I



antialiased



Anti-Aliasing

Typen

- Supersampling Anti-Aliasing (**SSAA**)
- Multisample Anti-Aliasing (**MSAA**)
- Fast Approximate Anti-Aliasing (**FXAA**)
- Temporal Anti-Aliasing (**TAA**)
- Morphological Anti-Aliasing (**MLAA**)
- Subpixel Morphological Anti-Aliasing (**SMAA**)
- Deep Learning / AI-based Anti-Aliasing (DLSS/FSR Super Resolution + Anti-Aliasing hybrids)

Anti-Aliasing

Supersampling Anti-Aliasing (SSAA)

- Erhöhung der Auflösung der Samples
- Downscaling vom Ergebnis

Praktisch: Der Frame wird bei einer höheren Auflösung gerendert und auf die Monitor Auflösung runterskaliert.

Full-resolution SSAA: Die ganze Szene wird mit einer höheren Auflösung gerendert

Anti-Aliasing

Temporal Anti-Aliasing (TAA)

Temporal Anti-Aliasing (TAA)

- Sammlung von Bilddaten über mehrere Frames
- Reduzierung von Aliasing mit historischen Daten
- Braucht wenig Rechenleistung

Probleme:

- Ghosting durch schnelle Bewegungen oder veraltete historische Daten
- Potentieller Verlust von kleinen Details
- Erhöhter Speicherverbrauch durch das Verwalten von historischen Daten

Anti-Aliasing

Temporal Anti-Aliasing (TAA)



Figure 34: Ghosting in Forza Horizon 5

Anti-Aliasing 🚫

Temporal Anti-Aliasing (TAA)



Figure 35: Frame aus dem Trailer von *Assetto Corsa Rally*. Man beachte die **Reifen**.



Figure 36: Ghosting der Reifen durch die schnelle Bewegung des Fahrzeugs.

Anti-Aliasing

Weitere Details folgen

Anwendung

Anwendung

- Entertainment
 - Filme
 - Videospiele
- Architektur Walk-throughs
- Datenvisualisierung
- Robotik
- Produktentwicklung
 - Autoindustrie
- Trainingssimulationen
 - Flugsimulator
 - Militär
 - Fahrschule

Anwendung

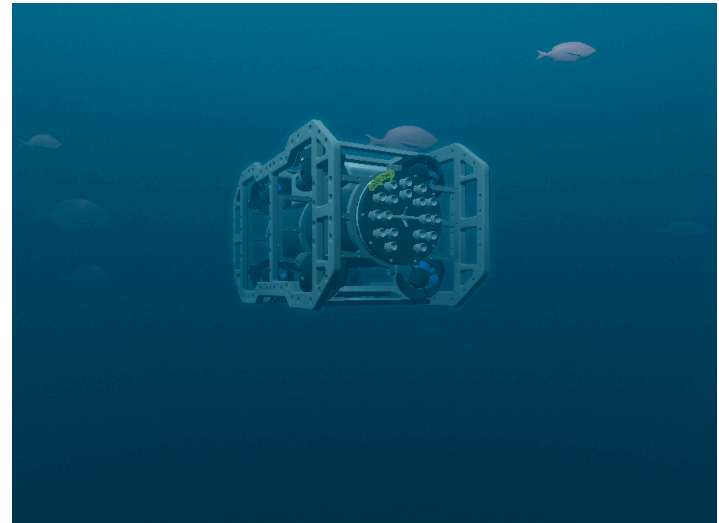
- Entertainment
 - Filme
 - Videospiele
- **Architektur Walk-throughs**
- Datenvisualisierung
- Robotik
- Produktentwicklung
 - Autoindustrie
- Trainingssimulationen
 - Flugsimulator
 - Militär
 - Fahrschule



Figure 37: Virtuelle Darstellung eines Umgebindehauses (IZU)

Anwendung

- Entertainment
 - Filme
 - Videospiele
- Architektur Walk-throughs
- Datenvisualisierung
- **Robotik**
- Produktentwicklung
 - Autoindustrie
- Trainingssimulationen
 - Flugsimulator
 - Militär
 - Fahrschule



Anwendung

- Entertainment
 - Filme
 - Videospiele
- Architektur Walk-throughs
- Datenvisualisierung
- Robotik
- Produktentwicklung
 - Autoindustrie
- Trainingssimulationen
 - Flugsimulator
 - Militär
 - Fahrschule

